Cahier des charges techniques

Sommaire

1. Contexte du projet
    1.1. Présentation du projet
    1.2. Date de rendu du projet
2. Besoins fonctionnels
3. Ressources nécessaires à la réalisation du projet
    3.1. Ressources matérielles
    3.2. Ressources logicielles
4. Gestion du projet
5. Conception du projet
    5.1. Le front-end
        5.1.1. Wireframes
        5.1.2. Maquettes
        5.1.3. Arborescences
    5.2. Le back-end
        5.2.1. Diagramme de cas d'utilisation
        5.2.2. Diagramme d'activités
        5.2.3. Modèles Conceptuel de Données (MCD)
        5.2.4. Modèle Logique de Données (MLD)
        5.2.5. Modèle Physique de Données (MPD)
6. Technologies utilisées
    6.1. Langages de développement Web
    6.2. Base de données
7. Sécurité
    7.1. Login et protection des pages administrateurs
    7.2. Hachages des mots de passe avec Bcrypt
    7.3. Protection contre les attaques XSS (Cross-Site Scripting)
    7.4. Protection contre les injections SQL

1. Contexte du projet

1.1. Présentation du projet

La Direction des Systèmes d'Information (DSI) de la préfecture souhaite faire évoluer l'application de gestion des cartes grises. En l'absence de documentation, votre mission est d'élaborer les documents de conception de l'application actuelle pour faciliter la réflexion autour de l'évolution du système actuel, puis d'implémenter de nouvelles fonctionnalités.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 15/01/2026.

2. Besoins fonctionnels

L'application devra disposer d'une interface permettant de consulter et de gérer l'ensemble des données relatives aux cartes grises.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via les différents formulaires de l'application, permettant aux utilisateurs d'administrer les propriétaires, les véhicules et les possessions.

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

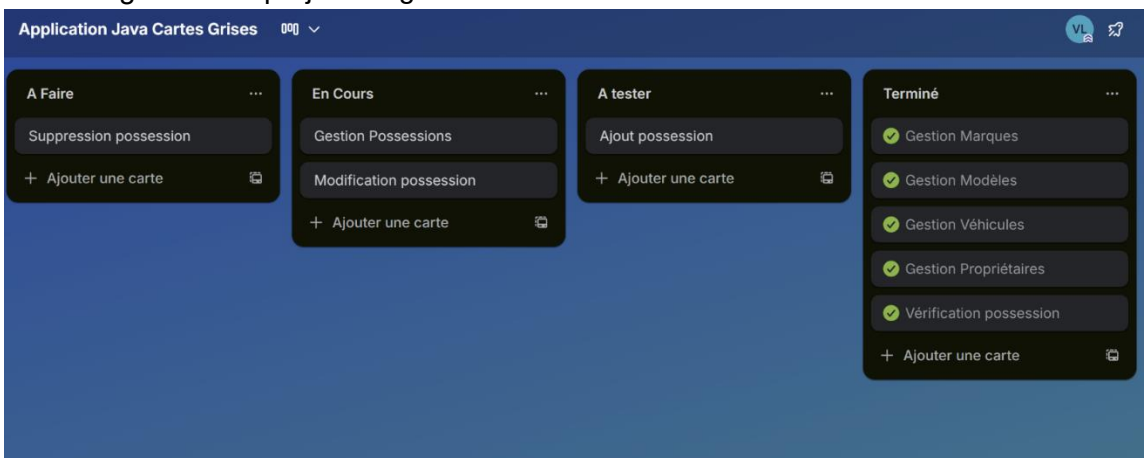
- PC
- Connexion Internet
- Serveur local
- Périphériques de travail

3.2. Ressources logicielles

- Visual Studio Code
- MAMP (contient MySQL)
- Mocodo

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.



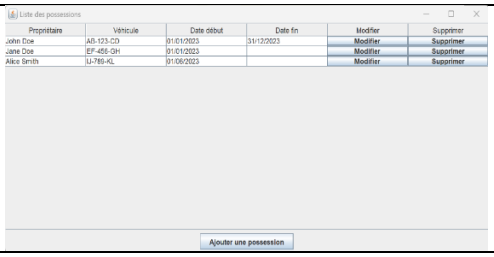
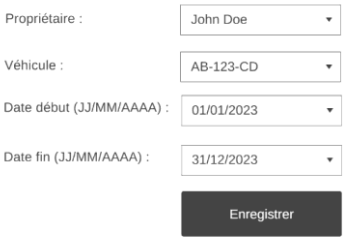
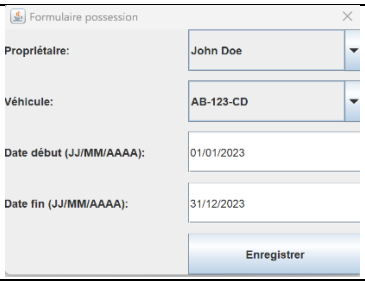
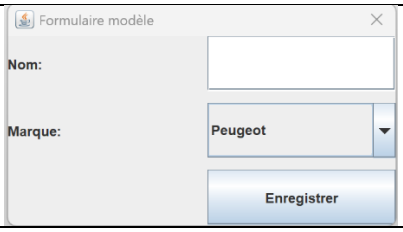
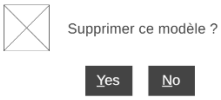
Nous travaillons également sur GitHub, plateforme de développement collaboratif.

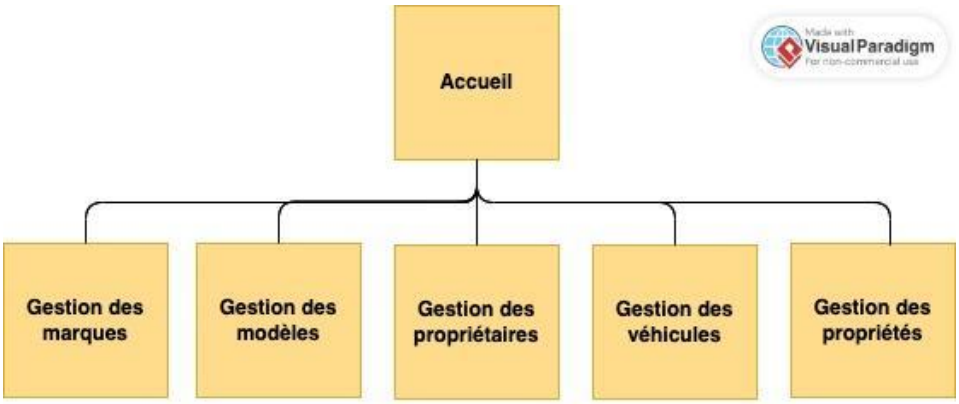
https://bit.ly/4rqxyHs

5. Conception du projet

5.1. Le front-end

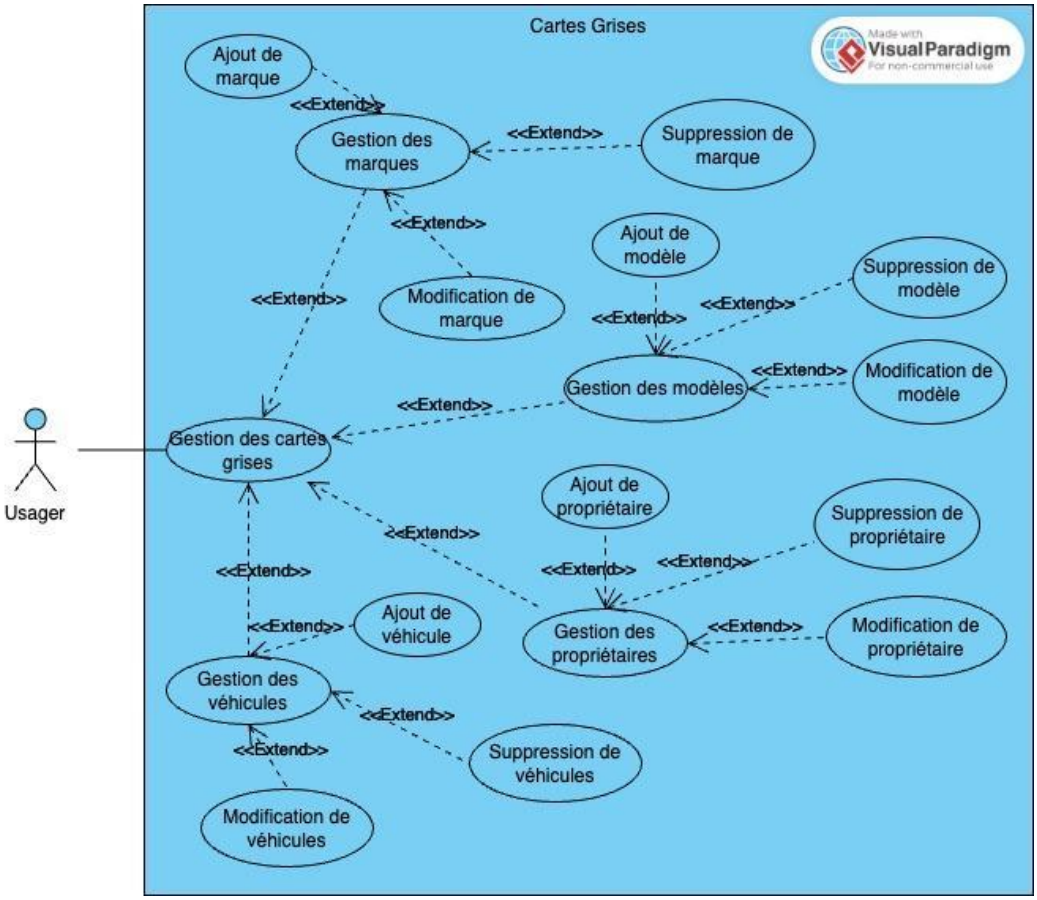
5.1.1. Wireframes et maquettes

Wireframe	Maquette
Fenêtre Accueil	
Fenêtre Liste Possessions	
	
Fenêtre Modification Possession	
	
Fenêtre Ajout Modèle	
	
Fenêtre Suppression Modèle	
	

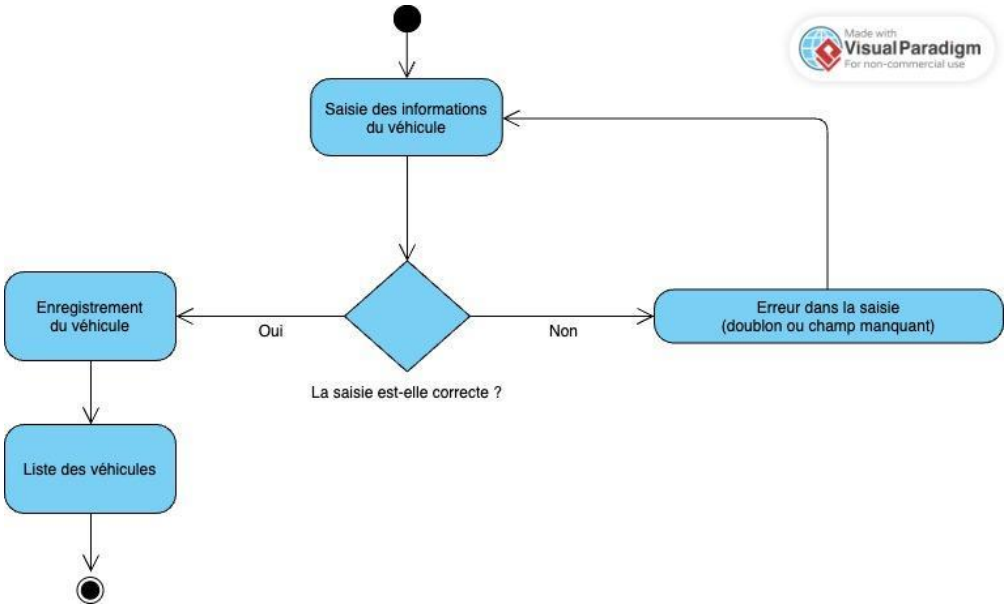


5.2. Le back-end

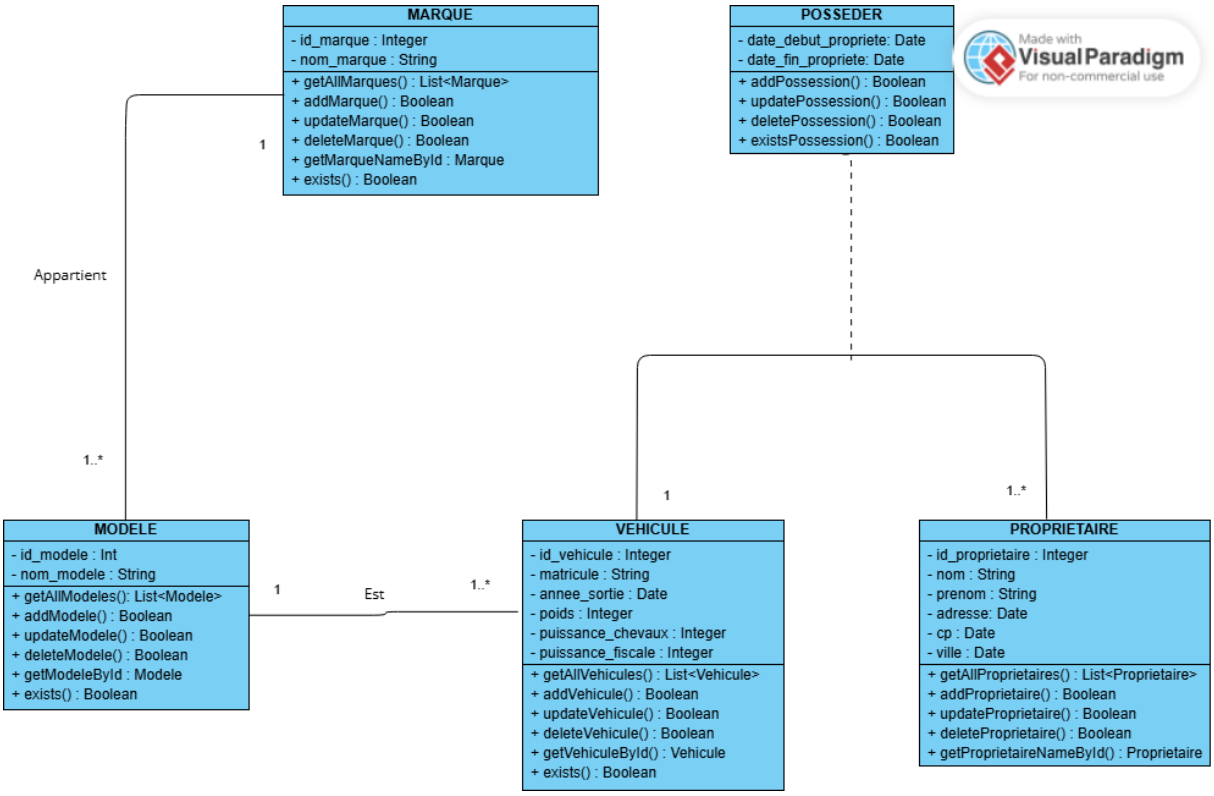
5.2.1. Diagramme de cas d'utilisation



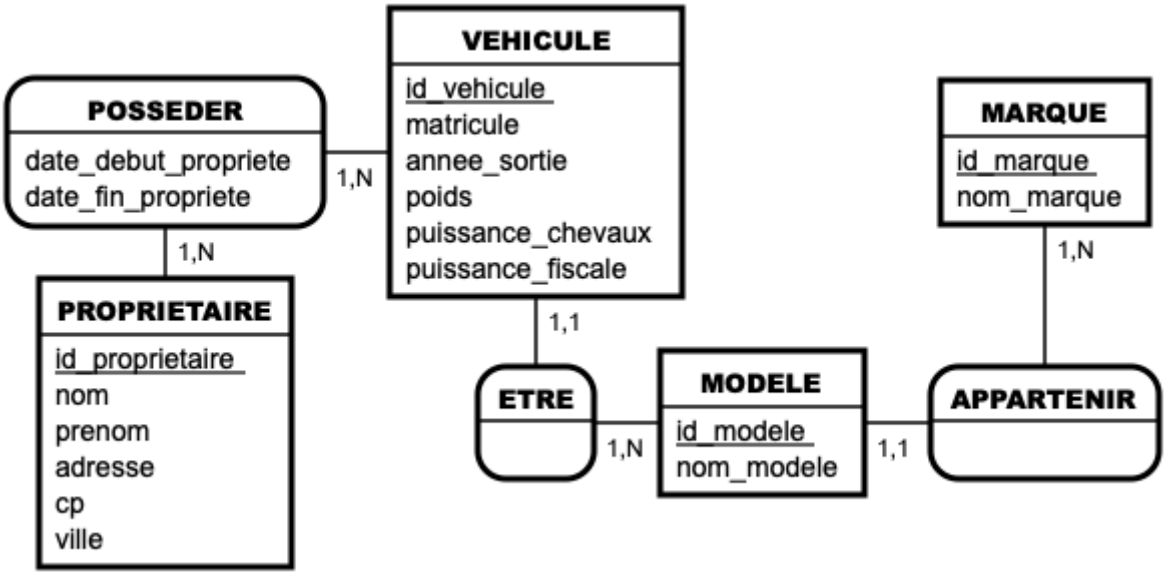
5.2.2. Diagramme d'activités



5.2.3. Diagramme de classe



5.2.4. Modèles Conceptuel de Donnée



5.2.5. Modèle Logique de Données (MLD)

● PROPRIETAIRE (id_proprietaire, nom, prenom, adresse, cp, ville)
Clé primaire : id_ proprietaire

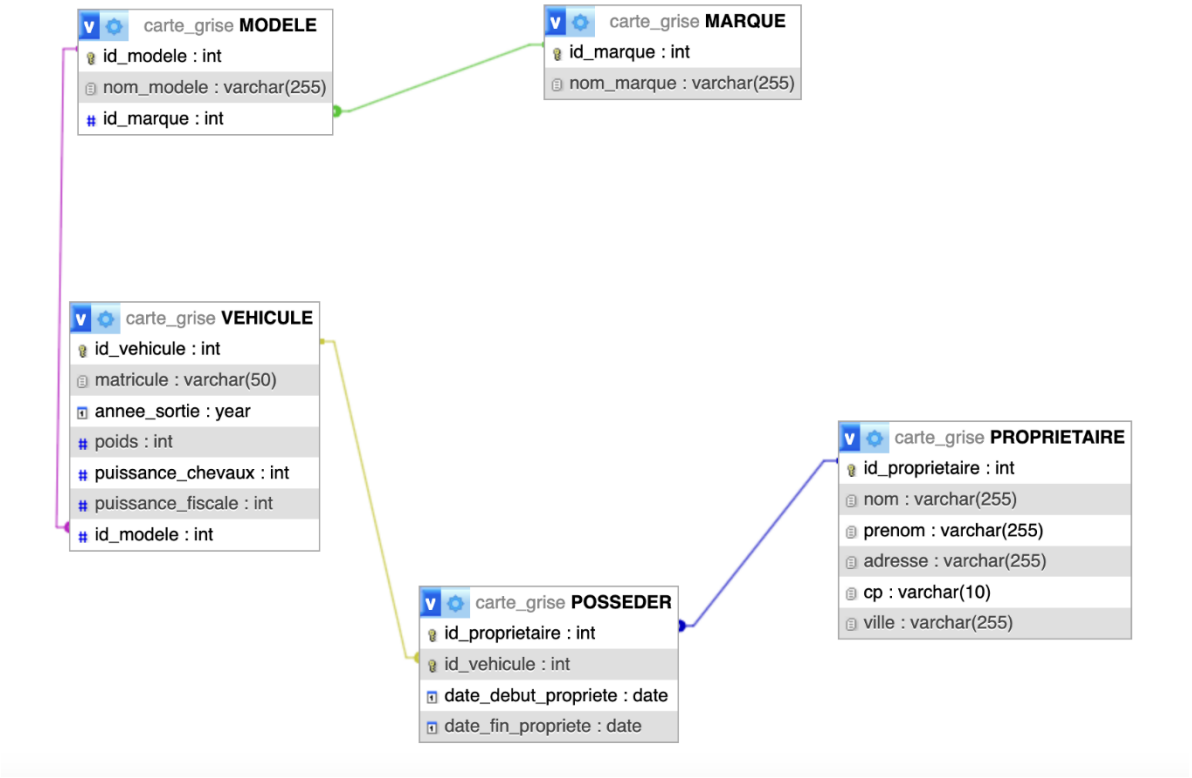
● POSSEDER (id_proprietaire, id_vehicule, date_debut_propriete, date_fin_propriete)
Clé primaire : id_proprietaire, id_vehicule
Clés étrangères : id_proprietaire en référence à id_proprietaire de PROPRIETAIRE
id_vehicule en référence à id_vehicule de VEHICULE

● VEHICULE (id_vehicule, matricule, annee_sortie, poids, puissance_chevaux, puissance_fiscale, id_modele)
Clé primaire : id_vehicule
Clé étrangère : id_modele en référence à id_modele de MODELE

● MODELE (id_modele, nom_modele, id_marque)
Clé primaire : id_modele
Clé étrangère : id_marque en référence à id_marque de MARQUE

● MARQUE (id_marque, nom_marque)
Clé primaire : id_marque

5.2.6. Modèle Physique de Données (MPD)



6. Technologies utilisées

6.1. Langage de développement

Java

6.2. Base de données

MAMP MySQL SQL*

*Structured Query Language

7. Sécurité

7.1. Protection contre les injections SQL

Dans une application Java, une injection SQL survient lorsqu'un utilisateur saisit des fragments de code SQL malveillants dans un champ de texte. Si ces données sont concaténées directement dans une chaîne String pour construire une requête, elles peuvent manipuler la base de données pour consulter, modifier ou supprimer des données sans autorisation.
Pour prévenir ces attaques, il est impératif d'utiliser des requêtes préparées via l'objet PreparedStatement. Cette méthode permet de séparer la structure de la requête SQL des données saisies par l'utilisateur. Ainsi, le moteur de base de données traite les entrées uniquement comme du texte brut, garantissant qu'elles ne soient jamais interprétées comme des ordres ou des commandes SQL.

```
public static boolean existsPossession(int idProprietaire, int idVehicule) {
        String sql = "SELECT COUNT(*) FROM POSSEDER WHERE id_proprietaire=? AND
id_vehicule=?";
        try (Connection conn = DBConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, idProprietaire);
            ps.setInt(2, idVehicule);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return rs.getInt(1) > 0;
            }
        } catch (SQLException e) {
            System.err.println("Erreur existsPossession : " + e.getMessage());
        }
        return false;
    }
```

Utilisation d'une requête préparée avec des paramètres de position lors de la vérification d'existence d'une possession

Dans ce cas on utilise une requête préparée par exemple pour vérifier l'existence d'une possession, mais cela s'applique à n'importe quelle requête SQL de l'application. Ici on utilise deux paramètres de positions pour associer les données de la table POSSEDER à des paramètres avec le « ? ». Puis c'est avec l'objet PreparedStatement qu'on va lier les paramètres de la méthode Java qui exécute la requête SQL ici existsPossession(), aux emplacements réservés « ? » dans la requête SQL.